GENERAL-PURPOSE ARCHITECTURE / ENGINEERING WHITE PAPER

The In-Isolate Software Workshop

A generative software substrate for agentic coding, generative UI, and measured swarm optimization

Software creation becomes a runtime operation.

Generate → type-check → execute → measure → improve → activate

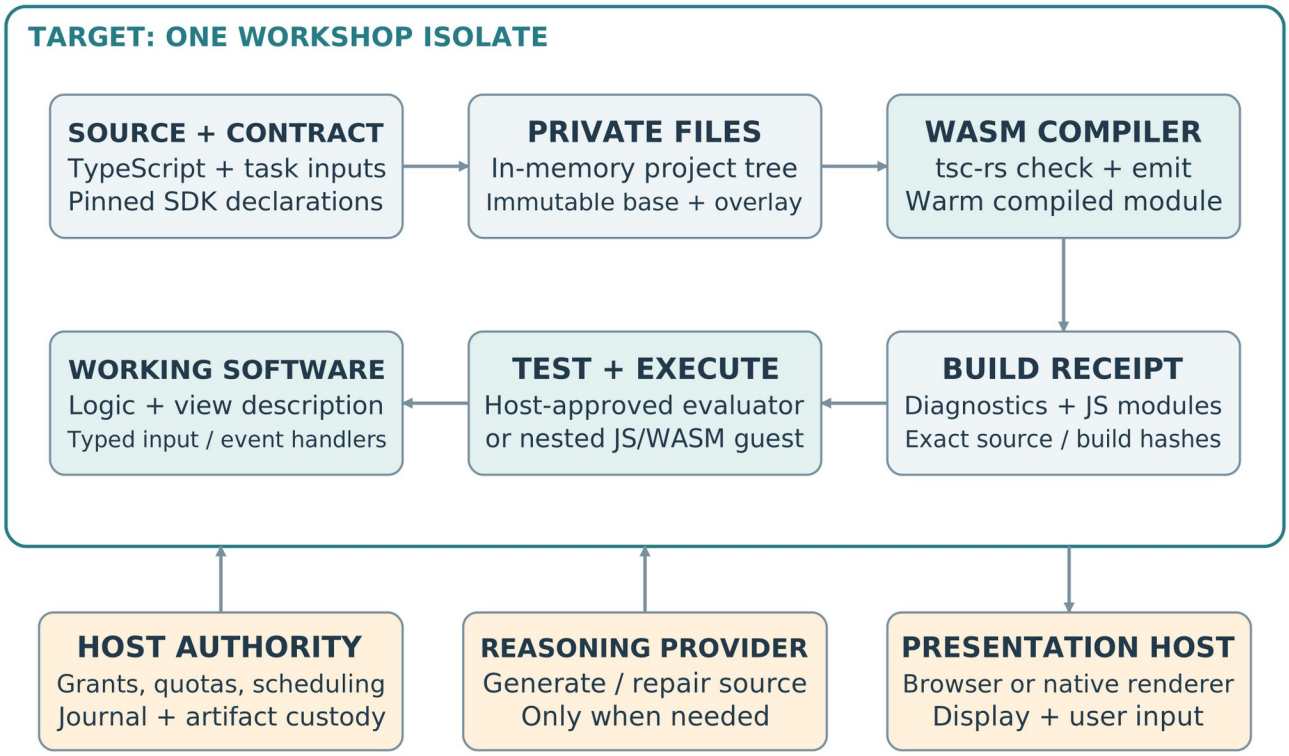


Figure 1. The target is an in-isolate build-and-execute workshop. Host authority, durable storage, reasoning, and display are explicit interfaces, not duplicated environments.

THE CENTRAL THESIS

Load the machinery once. Create small, isolated software candidates around the task. Keep only validated improvements, and release execution resources when there is nothing to compute.

Prepared 14 October 2026 • Version 1.0 • Audience: senior software engineers and technical leaders

Field OS is a concrete application and evidence source—not the definition of the substrate. Local results are reported from October 8 receipts; compiler results are upstream reports dated October 13. No new runtime experiment was performed for this document.

01 / THESIS / PROPOSED

The concept and the bigger picture

The proposal is a **general-purpose runtime that can manufacture useful software while an application is running**. A task-scoped workshop contains virtual files, a TypeScript compiler compiled from Rust to WebAssembly, a controlled evaluator, tests, and instrumentation. An agent supplies source or a patch; the workshop checks, emits, executes, and measures it; the host presents or activates the resulting capability. The unit of delivery becomes a small, versioned software capsule, not necessarily a separately provisioned application.

The strongest form places the build-and-test loop **inside one outer isolate**. Source need not cross a remote build service, compiler process, filesystem checkout, and deployment pipeline for every revision. The runtime retains expensive machinery while recreating mutable task state. An isolate pool extends the same pattern to many candidates or agents. This is a proposed synthesis; the Field OS implementation demonstrates selected components rather than the complete loop. [R1, R2]

Why this is more than an implementation optimization

A lower marginal cost of creating software changes which problems are worth solving with software. A one-off investigation can receive a purpose-built comparator. A user can ask for a new interaction and receive working logic rather than an explanation of how to build it. A coding agent can try alternative implementations against a fixed evaluator. An application can keep a useful generated module and reuse it without another model call on every interaction.

The progression is: **model output becomes executable behavior; executable behavior becomes a measured candidate; a passing candidate becomes a reusable capability**. Generative UI is one presentation of that capability. Swarm hill climbing is one method for improving it. The runtime substrate underlies both; it is not tied to refrigeration, Field OS, one model provider, or one UI framework.

What “everything inside” means

The compiler, virtual project, compatible tools, generated logic, local tests, and measurement hooks can inhabit the workshop. That does not require placing model weights, operating-system services, a browser compositor, credentials, or the authoritative evaluator in the same trust domain. Their interfaces remain explicit. A server isolate can create UI descriptions or application modules; a browser or native host still owns display and input. [R1, U4]

THE POTENTIALLY TRANSFORMATIVE CLAIM
Software can become an on-demand, continuously improvable part of ordinary work—not just a product built and deployed before that work starts.

That is a credible architectural opportunity, not yet a demonstrated universal speedup or a claim that the underlying primitives are new. Cloudflare already offers closely related dynamic execution and agent primitives. The differentiator must be the complete, measured product loop: usefulness, responsiveness, correctness, resource efficiency, and safe reuse. [U6, U7, U9, U10]

02 / ORIENTATION

Reading guide, terminology, and evidence

Reading route	Sections
Understand the idea	1, 3–9: concept, in-isolate architecture, agent loops, generative UI, swarm improvement
Assess the speed and resource case	10–12: cost model, native compiler benchmarks, WASM embedding
Evaluate runtime and cloud choices	13–14 and 20: loading topologies, Cloudflare implications and prior art
Inspect existing evidence	15–18: Field OS implementation, timings, containment, recovery and test scope
Plan implementation and qualification	19–22: contracts, operating model, benchmark campaign, implications

Terms used precisely

Term	Meaning in this document
Workshop isolate	A task- or trust-scoped execution context containing build/check/run machinery.
WASM module / instance	The reusable compiled program versus one instance's mutable linear memory and state.
Software capsule	Immutable code plus schemas, dependency identities, capability requirements, tests, and provenance.
Agent / attempt	A durable task and decision history versus one bounded period of actual execution.
Swarm	A budgeted population of candidate-producing workers; not necessarily simultaneous model inference.
Hill climbing	Iterative candidate selection against declared objectives, with gates, holdouts, and diversity.
Generative UI	Generated working presentation and behavior, executed by normal code after generation.

Evidence legend

Observed—repository receipt: a recorded Field OS experiment or test result. Raw private machine receipts were not rerun or independently remeasured for this document. **Upstream-reported:** a maintainer's documentation or benchmark, pinned where practical. **Derived:** arithmetic calculated from stated values. **Proposed:** architecture, interfaces, targets, or experiments not yet established by the receipts.

Bracketed references distinguish repository sources [R1–R9] from upstream sources [U1–U10]. Diagrams describe the proposed substrate unless labeled as the observed Field OS adapter. "tsc-rc" in the discussion is treated as **tsc-rs**, the ts-rust compiler named in the inspected source. [U1]

The document does not conflate three different comparisons: native compiler speed, QuickJS adapter warm-up behavior, and the end-to-end performance of an in-isolate compiler/evaluator/UI loop. Only the first two currently have the cited measurements.

03 / ARCHITECTURE / PROPOSED

The complete workshop inside the isolate

The hot path is a local transformation from an admitted source packet to a tested module. It starts with a small, explicit SDK and a pinned project environment. A compiler instance sees a private virtual filesystem rather than arbitrary host paths. Compilation yields diagnostics and JavaScript; execution produces typed results, view definitions, or candidate patches. [R1, U2, U3]

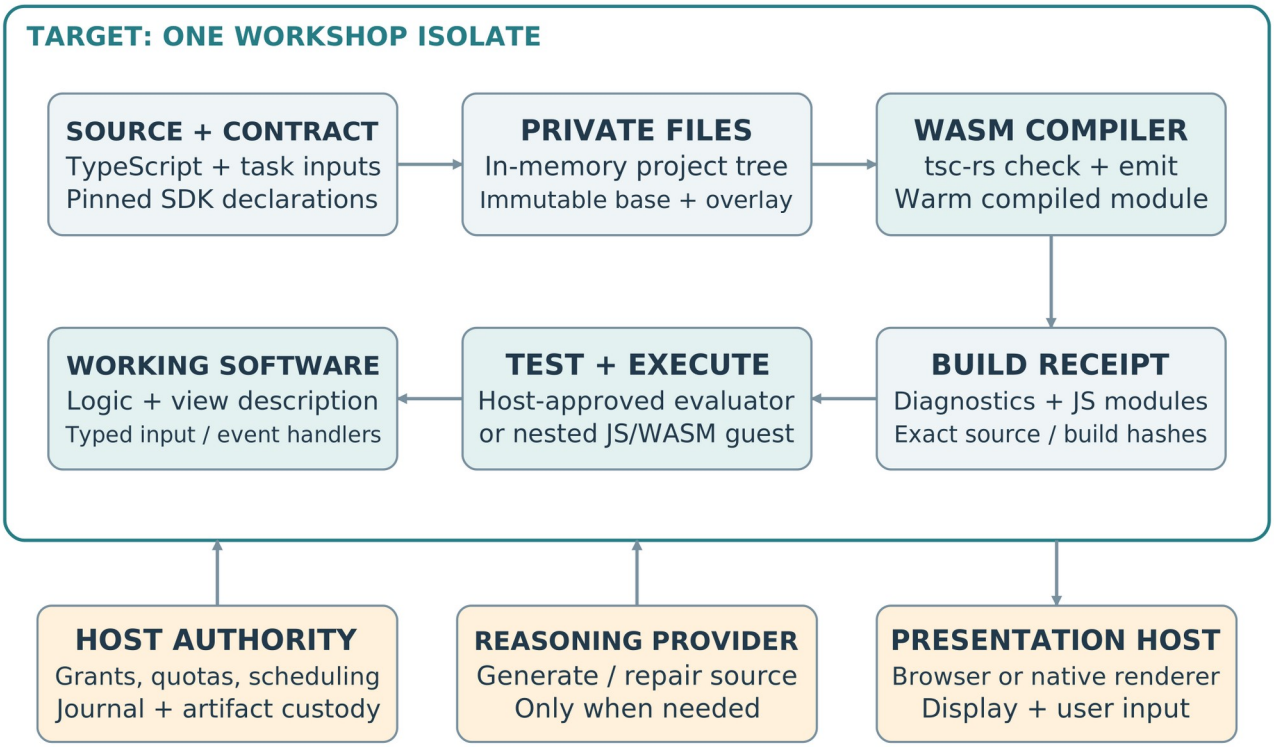


Figure 2. In-isolate target topology. Local fixture tests can run inside; authoritative scoring, permissions, and promotion remain outside candidate control. The choice of evaluator is a host-specific implementation decision. [R1, U2, U3]

One semantic loop; two distinct compiled programs

Rust → WASM produces the TypeScript checker/emitter that the workshop loads. **TypeScript → JavaScript** produces the candidate application. The candidate does not become a Rust application because its compiler is written in Rust. V8 can host the WASM compiler and compatible JavaScript; a nested QuickJS/WASM evaluator is another way to execute generated JavaScript within the same outer isolate. [U1–U3]

The crucial reuse is the runtime and compiled WASM machinery. Fresh compiler instances still create ASTs, symbols, type graphs, and output buffers. A warm WASM module is not automatically an incremental TypeScript program. The inspected WASM interface creates a new instance for each run and does not support watch mode. [U2]

04 / ARCHITECTURE / PROPOSED

The runtime fabric around the workshop

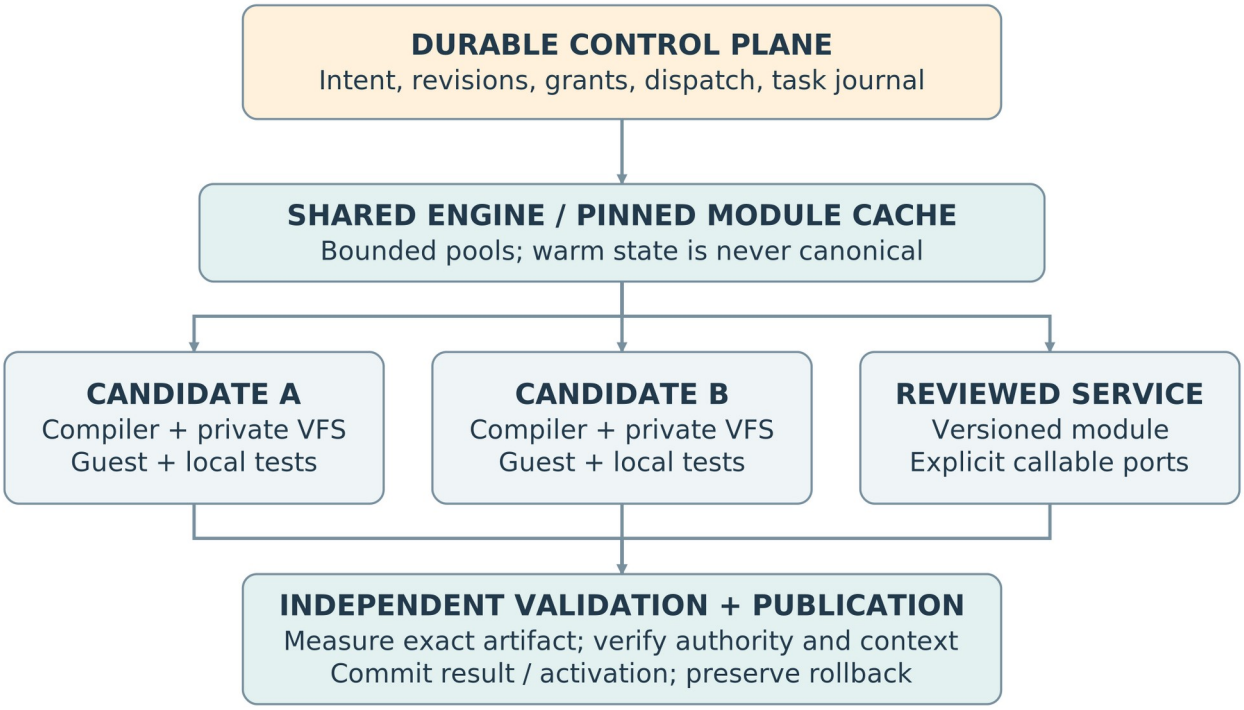


Figure 3. A bounded fabric can schedule many logical tasks onto a smaller number of active workshops. Sharing engine machinery does not imply shared mutable candidate heaps or independent process-failure boundaries. [R1, R2]

Responsibilities a senior engineer should preserve

The **control plane** owns task identity, current grants, queueing, admission, execution permits, resource profiles, and state transitions. The **artifact store** owns immutable code, input packets, tests, build receipts, and outputs. The **execution plane** runs compatible computation. The **presentation host** renders approved primitives or separately sandboxed modules. None of these roles needs to become a new daemon simply because it appears as a box in the diagram. [R1]

A trusted native adapter remains available for browser automation, native packages, full POSIX behavior, GPU work, or a library that cannot fit the chosen WASM/Workers ABI. That is an explicit escape to a different execution class, not an assertion that every existing npm package or application will run unchanged inside an isolate. [R1, R2]

The common interface is a capsule plus a capability contract. A browser, local workstation, site server, and managed cloud can implement that contract differently. Avoid embedding filesystem paths, live credentials, renderer object handles, or cloud-specific service identities into portable application logic.

05 / ARCHITECTURE / PROPOSED

Reuse the machinery, not the mission heap

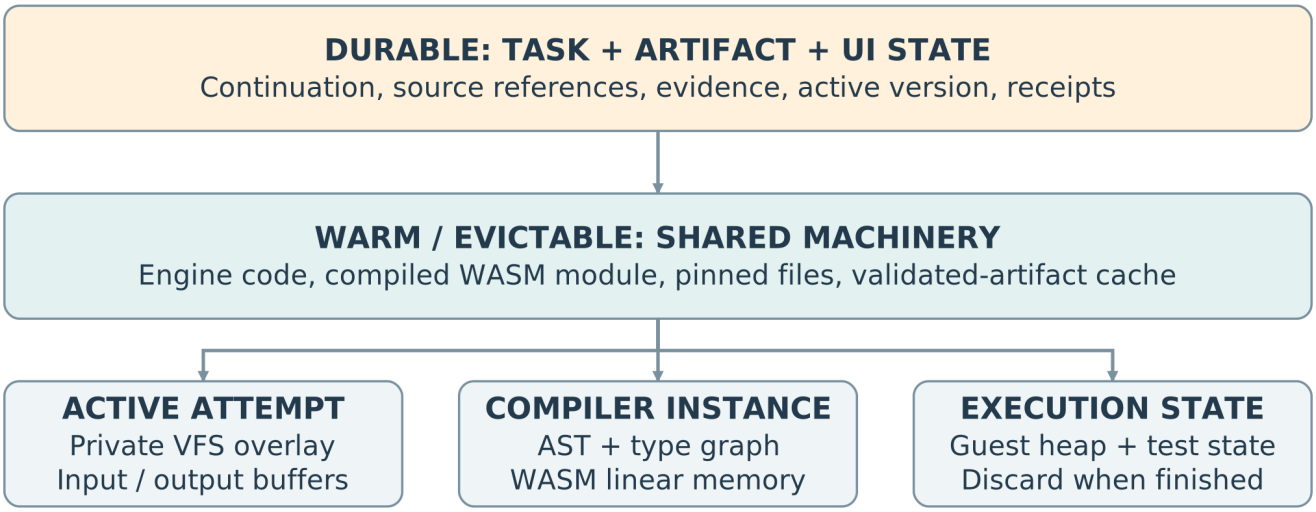


Figure 4. Three lifetimes: durable task state, evictable warm machinery, and disposable attempt state. This separation enables high logical task counts without one resident environment per task. [R1, R2, R3, U2]

An agent waiting for a model, user, or network response can persist its next step and release its execution slot. On resume, the scheduler reconstructs a bounded attempt from immutable references and revalidates grants. A live Promise is not a durable task record. UI state that must survive restart also belongs in the host's versioned state model, not only in a guest heap. [R1, R5]

A practical cache hierarchy

Reusable object	Key / invalidation rule	Isolation rule
WASM compiler module	Compiler bytes + runtime/feature identity	Reuse compiled code; separate mutable instances
Pinned project base	Dependency graph + SDK declarations + configuration	Read-only base; private writes per candidate
Checked build output	Source + compiler + options + dependency digests	Cache diagnostics and exact output together
Measured result	Artifact + inputs + evaluator + policy + environment	Do not reuse across incompatible source scope
Active UI version	Artifact + state schema + current grants	State migration and activation are governed

Fresh-state execution does not prohibit efficient reuse. The October 8 warm trials retained host machinery while one arm recreated the VFS per job; its median was essentially the same as the shared-VFS arm for that specific fixture. That result supports testing this pattern, not a general proof of zero copy cost or zero cross-job leakage. [R3]

Module sharing across separate isolates or host processes is runtime-dependent. Do not assume a single physical copy of every compiled module, declaration file, or AST across the whole machine. Cache residency, retained memory, and eviction behavior must be measured.

06 / ARCHITECTURE / PROPOSED

Agentic coding: turn reasoning into a fast inner loop

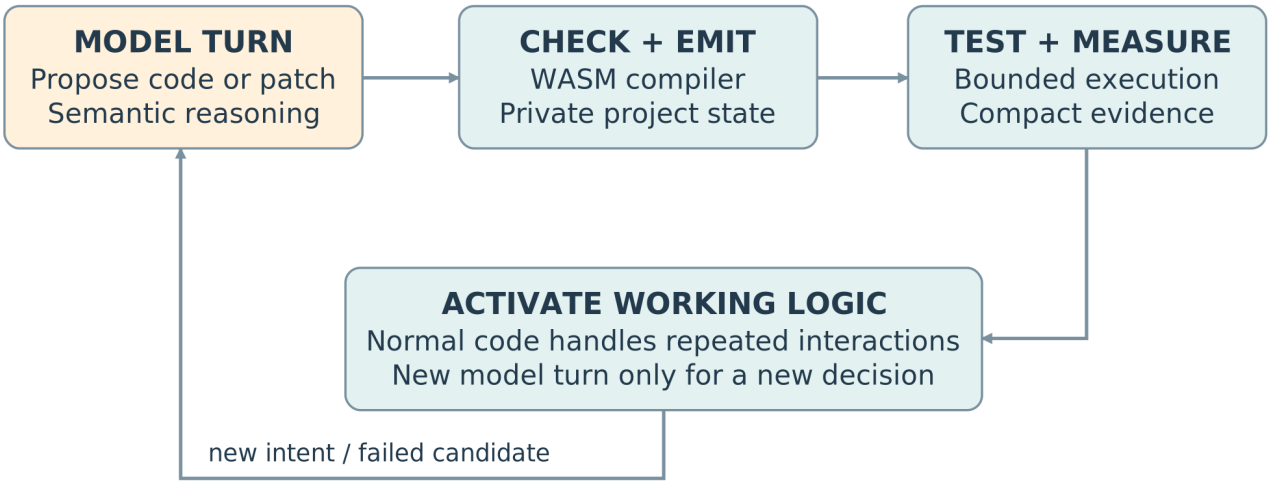


Figure 5. Reasoning supplies or repairs a candidate; the workshop performs deterministic checking and execution. Repeated user interactions call working code rather than regenerating the application. Proposed workflow, informed by Code Mode. [R1, U6]

A model is valuable at selecting an approach and proposing new logic. It is not necessary for every branch, loop iteration, filter, or API-result transformation. A generated program can compose permitted tool calls, handle intermediate data, and return compact evidence. Cloudflare documents this as Code Mode; it is a useful precedent for keeping deterministic orchestration out of repeated model turns. [U6]

The unit of iteration becomes a patch, not an environment

For each candidate, apply a delta to the private project tree, type-check against the actual SDK, emit exact JavaScript, run bounded tests, and return structured diagnostics. Retain dependency machinery and the compiled checker. Do not reinstall packages or respawn a full build toolchain simply because a generated function changes. This is the target architecture, not the current Field OS gateway, which still accepts a prebuilt self-contained script. [R6, R7]

The agent can then use the compiler as rapid feedback: missing method, wrong parameter, incompatible state shape, or invalid result contract. Type checking prunes structurally invalid candidates before expensive behavioral or UI tests. Semantic correctness still requires independent tests and source validation; a type-correct hallucinated formula remains wrong.

Reason once, execute many times

A useful comparator, parser, mapper, or interaction handler can be reused after generation. The cost of reasoning and checking is amortized across subsequent executions. New reasoning is needed for new intent or unresolved behavior—not necessarily for every click. This may reduce model round trips and intermediate context, but a large search over candidate programs can also consume more tokens. Count total search cost, including failed candidates, rather than assuming a swarm is cheaper.

07 / ARCHITECTURE / PROPOSED

Generative UI becomes working software

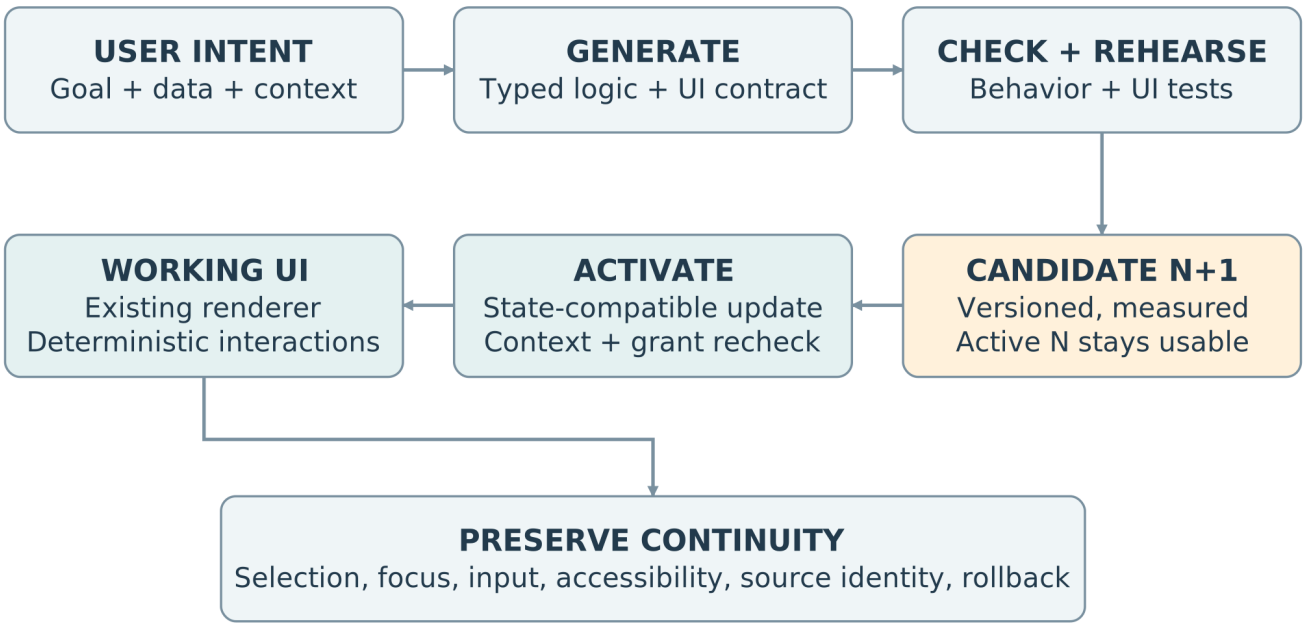


Figure 6. Generate a candidate UI version without mutating the active one. Validation and state-compatible activation turn generated output into a working instrument. Proposed general lifecycle; Field OS supplies source-binding requirements. [R1, R9]

The UI is no longer only a generated screen. A capsule can contain typed view structure, calculation logic, input validation, state transitions, sorting and filtering, and explicitly bound actions. The existing application supplies rendering, styling, accessibility, data access, and effect approval. Working code handles interaction after the initial generation.

For example, a user investigating inconsistent measurements could receive a comparison panel with unit controls, source links, uncertainty labels, and a discrepancy table. A later request could add a sensitivity plot or change the grouping. The host compiles and tests version N+1 while N remains usable, then activates the new version without losing the user's selection or source context. This is a proposed interaction, not a recorded Field OS UI result.

The stable contract is more important than the visual novelty

Use a versioned presentation grammar for common controls and views; use a separate, capability-limited application sandbox when the grammar is insufficient. Preserve identity, input focus, unsaved state, keyboard behavior, accessible names, and rollback. An updated layout must not silently reinterpret a number or redirect an action to a different object. Field OS's exact room/source bindings illustrate why UI state and domain authority must remain distinct. [R9]

UI improvement needs a real renderer test. Structural schema validity cannot establish legibility, navigation, responsiveness, or task completion. A candidate may be better because it exposes fewer misleading options—not because it renders more components. Keep high-consequence action approval in the host, outside generated presentation code.

08 / ARCHITECTURE / PROPOSED

Swarm hill climbing: generation with measurement

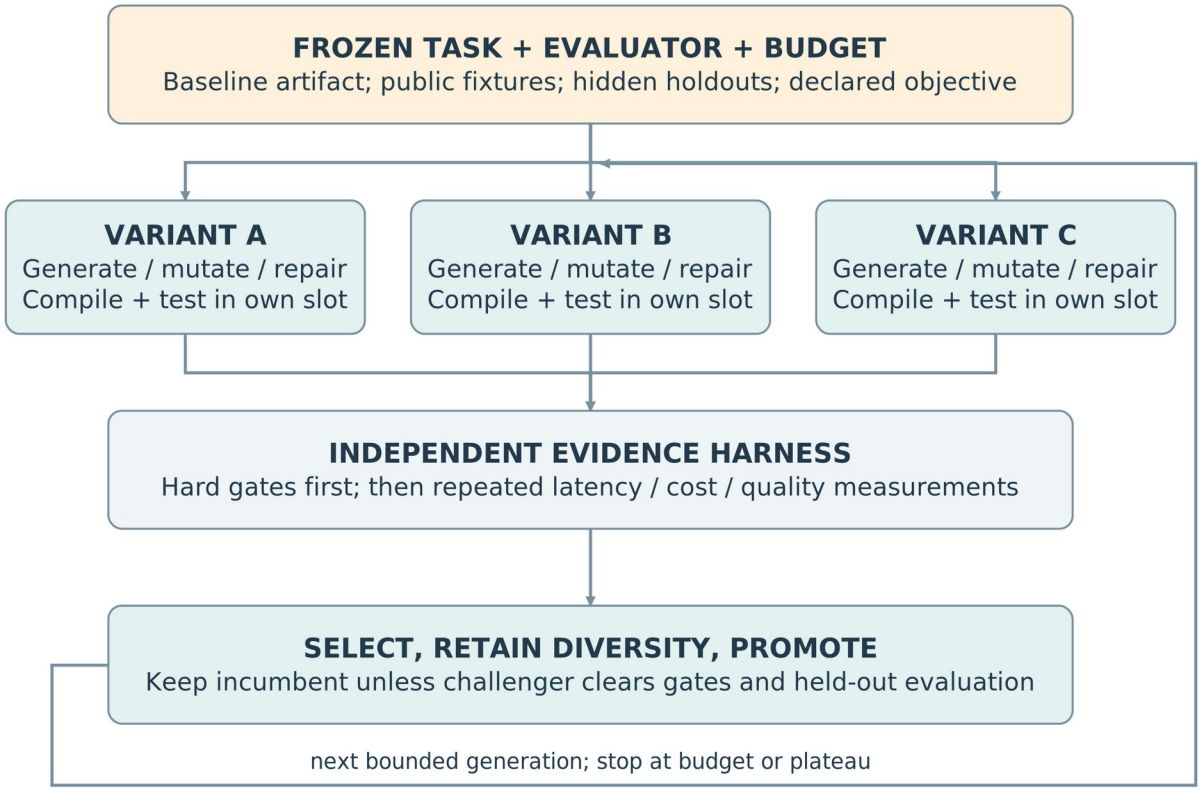


Figure 7. A bounded search population generates candidates inside isolated workshops. An independent harness measures exact artifacts; selection preserves an incumbent and a budget. This complete swarm loop is proposed, not yet measured in the Field OS receipts.

A swarm does not need a separate operating environment per agent. It needs independent candidate state, traceable lineage, and enough execution slots for the available CPU, memory, and inference budget. The runtime can keep many candidate records while only a few workers are active. WASM checking can reject malformed variants before more expensive evaluation.

Each generation starts from an incumbent and selected diverse parents. Workers propose different algorithms, data structures, view layouts, or task strategies. The harness first enforces hard correctness and policy gates, then measures objective quality, latency, CPU, memory, and user-task outcomes. A challenger replaces the incumbent only when it clears the declared acceptance rule.

Plain hill climbing can plateau or overfit its evaluator. Preserve diversity, use restarts or alternate objectives when justified, and keep untouched holdout tasks. More candidates increase the chance of finding both a useful improvement and a lucky measurement. Repeated trials and held-out evaluation are therefore part of the architecture —not optional reporting polish.

09 / EVALUATION / PROPOSED

Build a trustworthy optimization loop

Separate hard constraints from optimization objectives

A performance improvement cannot compensate for wrong answers, broken accessibility, data leakage, or unauthorized effects. Evaluate validity first. Among eligible candidates, compare a vector of outcomes rather than hiding all tradeoffs inside one score. A faster implementation that doubles memory may be suitable on a workstation and unsuitable on a small appliance.

Stage	Required evidence	Who controls it
Contract gate	Source/build identity, schema compatibility, capability ceiling	Trusted controller
Behavior gate	Independent assertions, invariants, regression fixtures	Evaluator outside candidate scope
Resource gate	Deadline, peak memory, CPU, output volume, cancellation	Host / kernel / managed runtime
UI gate	Task completion, interaction parity, accessibility, state retention	Renderer harness and human review
Selection	Repeated comparisons; held-out confirmation; declared tradeoff	Frozen acceptance policy
Promotion	Exact artifact, proof bundle, current grants, rollback identity	Authority owner

An implementable search skeleton

```
incumbent = load_verified_baseline(task_revision)
while budget.remaining() and not stop_rule.reached():
    parents = select_diverse_parents(incumbent, archive)
    proposals = generate_bounded_variants(parents, public_feedback)
    for candidate in scheduler.admit(proposals):
        build = workshop.check_and_emit(candidate)
        if not build.contracts_pass: record_rejection(build); continue
        evidence = trusted_harness.evaluate(build.exact_artifact)
        archive.append(candidate, build, evidence)
    challenger = select_eligible_candidate(archive, frozen_objective)
    if heldout_accepts(challenger, incumbent):
        incumbent = commit_candidate(challenger, expected_revision)
return incumbent, complete_search_receipt
```

This is proposed orchestration pseudocode, not an existing API. The candidate must not alter the evaluator, scoring weights, hidden fixtures, timing boundaries, or its own host counters. Generated tests may supplement—but cannot replace—the independent acceptance oracle.

Guard against measurement-driven failure

Record seeds, code and input digests, runtime versions, CPU quota, warm/cold state, contention, and compiler mode. Randomize trial order and use paired comparisons where possible. Keep validation feedback useful but prevent repeated exposure from turning the holdout into training data. For noisy UI and task-quality objectives, require human evidence where automation cannot establish usefulness.

Stop on budget exhaustion, plateau, regression, or ambiguous execution. Keep the incumbent working throughout. A failed candidate is evidence, not permission to weaken the objective after seeing its result. Count all model, compiler, test, rendering, storage, and reviewer costs in the final search receipt.

10 / PERFORMANCE MODEL / DERIVED + PROPOSED

Where speed and resource advantages come from

There are three distinct levers: **amortize setup**, **shorten deterministic feedback**, and **remove unnecessary reasoning turns**. Shared machinery does not make CPU work free, and a compiler speedup is not automatically an application speedup.

```
T_cycle = T_model + T_queue + T_prepare + T_check
        + T_test + T_measure + T_publish

M_substrate ~= M_host + M_warm_pools
             + sum(M_private_files + M_compiler_instance
                   + M_guest + M_buffers) for active attempts
             + M_bounded_caches
```

The memory expression is an accounting model, not a measured allocation. Sequential phases may release compiler state before the guest reaches peak memory, so observed peak can be smaller than the sum; overlapping work and handoff copies can increase it. Containers already share a kernel and may share image/cache pages. Compare incremental resident memory and proportional set size where available, not a fictional full OS copy per container.

Mechanism	Potential advantage	Measurement that matters
Warm compiler / engine	Avoid repeated startup and module compilation	Cold start, first call, steady-state median and tail
Private VFS over pinned base	Avoid repeated checkout / dependency setup	Bytes copied, lookup cost, retained memory
In-isolate check/run	Avoid process launches and build-service round trips	Complete source-to-validated-artifact latency
Durable waiting	Do not reserve guest heaps for idle tasks	Resident bytes per active versus waiting task
Generated reusable logic	Avoid repeated reasoning for deterministic behavior	Model turns and total cost per accepted task
Budgeted parallel candidates	Reduce search wall time when resources allow	Accepted quality at equal total budget

Do not multiply unrelated benchmark ratios

If checking occupies fraction f of a fixed workflow and becomes s times faster, the idealized total speedup is **1 / ((1 − f) + f / s)**. At f = 0.60 and s = 10, that is about 2.17×; at f = 0.10, about 1.10×. Removing whole model or deployment steps can change the workflow itself and create additional gains, but those must be measured separately.

For swarm search, active throughput is bounded by CPU-seconds, memory headroom, compiler concurrency, model limits, and evaluator capacity. A single-threaded isolate does not turn many CPU-bound candidates into parallel CPU execution. Schedule them across suitable workers/processes or machines; do not make the UI event loop carry an unbounded compiler queue.

THE RIGHT UNIT OF SUCCESS
Measure time and total resources per accepted useful capability—not only checks per second, isolates created, or the fastest successful candidate.

11 / UPSTREAM-REPORTED / OCTOBER 13, 2026

tsc-rs: the gains and their exact scope

The inspected ts-rust repository reports native **tsc-rs 0.6.0** benchmarks on 16 AMD EPYC vCPU runners with roughly 62 GiB RAM. Each case used one warm-up and five measured runs. The T3 Code totals below are **sums of medians across five projects**, not elapsed time of one parallel workflow. They are not Field OS or WASM measurements. [U1]

T3 Code mode	TypeScript 7	tsc-rs	tsc-rs --fast
Without Effect diagnostics	19.45 s	7.95 s	6.91 s
With Effect diagnostics	42.55 s	14.30 s	9.77 s
Speedup vs corresponding TS7: no Effect	1.00×	2.45×	2.81×
Speedup vs corresponding TS7: Effect	1.00×	2.98×	4.35×

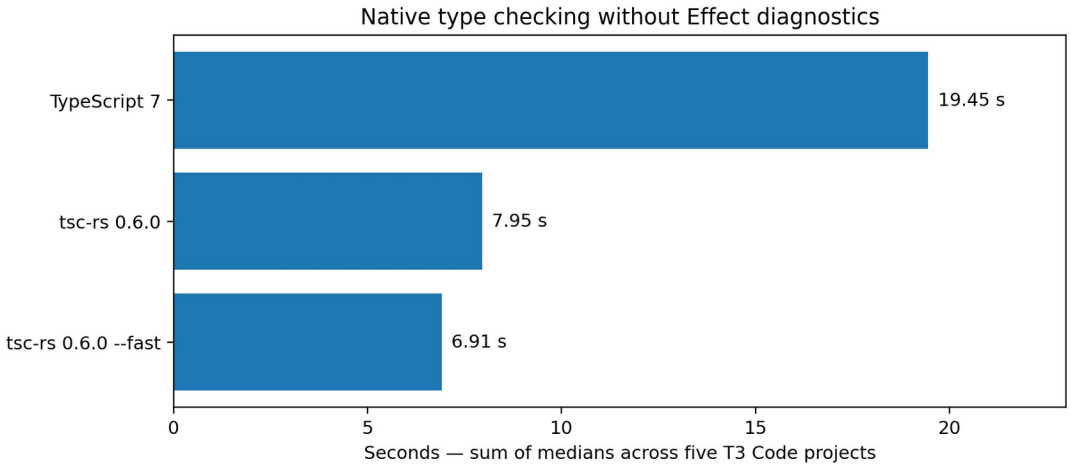


Figure 8. Native T3 Code checking without Effect diagnostics. Maintainer-reported medians, not in-isolate timings; the two Rust modes match the pinned TypeScript 7 diagnostics for this comparison. [U1]

Across six application benchmarks, the reported geometric-mean advantage versus TypeScript 7 is 2.10× in default mode and 2.90× in fast mode; versus TypeScript 6 it is 15.3× and 21.2×. Version changes and checking behavior matter. In the reported fixtures, Rust modes and pinned TypeScript 7 matched full diagnostics; Bun and TypeScript 6 did not always perform equivalent checks. With Effect, compiler versions and rule sets also differ. [U1]

The resource tradeoff is explicit: the maintainer reports fast mode using up to 2.2× CPU time and 1.9× peak memory in its 16-CPU measurements. These are not universally simultaneous gains in speed and memory. For a dense swarm, a lower-thread-count mode may yield better total throughput even when one check finishes later. [U1]

Implication: a fast checker can make repeated candidate validation economical. The missing bridge is a matched benchmark of the same checked projects through the WASM compiler inside the target isolate. Native ratios cannot simply be assigned to that path.

12 / UPSTREAM CAPABILITY + PROPOSED INTEGRATION

WASM turns the compiler into a runtime component

The inspected WASM package describes a **4.2 MB compiler module**, about 1.8 MB gzip or 1.5 MB Brotli, including compressed TypeScript libraries. A larger 5.2 MB build is reported to check 13–18% faster. The package documentation says publication is not enabled: this is a source-buildable interface, not a verified published WASM dependency to install blindly. [U2]

What the embedding surface actually supplies

Its runtime-independent core accepts a compiled WebAssembly module and a host filesystem interface. The high-level in-memory interface maps absolute virtual paths to file text and returns diagnostics and emitted files. The core supplies a small WASI shim and filesystem bridge; it does not require Node's WASI filesystem. This makes an in-isolate adapter plausible, but the inspected documentation does not establish a completed workerd integration. [U2, U3]

Property	Engineering consequence
Single-threaded WASM checker	Native multi-core / --fast gains do not transfer mechanically.
New instance for each compiler run	Reuse compiled module code; budget fresh type graphs and linear memory.
In-memory files and structured diagnostics	A generated patch can be checked without a host project checkout.
No watch, LSP, or API mode in this WASM interface	Do not describe a persistent incremental language service as already implemented.
Host-dependent stack behavior	Stress deep types and syntax on the actual runtime before admitting arbitrary projects.
Source-build publication status	Pin the build, module hash, SDK declarations, and compiler conformance target.

Binary size is not runtime memory

A 4.2 MB download does not imply a 4.2 MB isolate, heap, or project check. Include compiled machine code, WASM linear memory, AST/type graphs, source strings, duplicate encodings, emitted modules, VFS data, stacks, and host buffers. The Node wrapper's documented 256 MB worker stack setting is not evidence that every host commits that much memory; it is also not an available Workers setting by default. [U2]

The minimum embedding work

Bundle a pinned compiler module, supply an allowlisted virtual filesystem and SDK, adapt clocks/random/output imports as required, enforce source and depth limits, compare diagnostics and emitted bytes to a pinned reference, then connect a host-supported evaluator. Module resolution and bundling are separate obligations: tsc emission is not proof that runtime imports resolve. Lock dependencies; never let a generated candidate install arbitrary packages during execution.

Once this works, the application can treat compilation as an ordinary runtime capability. That is the enabling step for the proposed "everything inside the isolate" loop—not a claim that the whole operating system or inference engine fits inside it.

13 / PLATFORM IMPLICATIONS / VERIFIED SOURCES

Cloudflare: exact execution topology matters

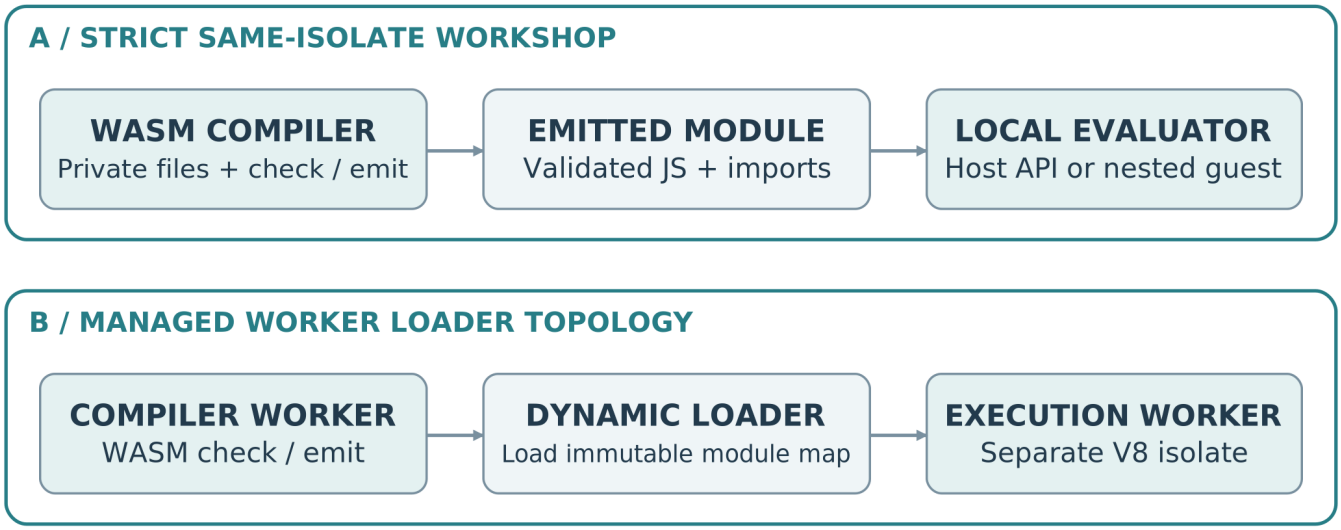


Figure 9. Two valid implementation shapes. A keeps build and execution in the same outer isolate. B uses a compiler Worker plus a separate execution isolate. B stays inside the Workers platform but is not literally one isolate. [U2, U3, U7, U8]

Strict in-isolate form: run the WASM checker, virtual files, and tests in the workshop. Generated JavaScript then uses an explicitly permitted host evaluation API or a nested JS/WASM interpreter such as QuickJS. The nested route maintains one outer isolate but introduces another engine and its overhead. Executing directly in a privileged realm is not an isolation boundary; type checking does not make it safe.

Managed Worker Loader form: a compiler Worker produces a module map and a loader creates an execution Worker. Current documentation distinguishes `load()` for a fresh Worker from `get(id, callback)` for potentially warm reuse. Content and configuration changes require a new identity, and cache residency is not guaranteed. [U7]

Cloudflare's Dynamic Workers playground already documents runtime TypeScript bundling, dynamic execution, and bundle reuse. That establishes relevant adjacent functionality, not full tsc-rs semantic type checking or the complete measured search loop proposed here. The loader consumes executable modules; checking and dependency preparation must happen before that handoff. [U10]

What must be verified on the chosen host

Do not rely on arbitrary request-time `eval`, assume Node worker-thread support, or feed raw WASM bytes into an unsupported compilation path. Cloudflare documents instantiation of precompiled WASM modules; the loader has its own module-loading surface. A successful import is not proof that the compiler adapter, stack behavior, or evaluator works in that environment. [U7, U8]

For generated code, explicitly block ambient egress and pass only narrowly scoped bindings. The current loader defaults to inherited outbound access when no override is specified; `globalOutbound: null` cuts off its global network access. Binding calls remain capabilities and must be governed separately. [U7]

14 / STRATEGY / ARCHITECTURAL INFERENCE

Cloud implications, prior art, and differentiation

What this implies for Cloudflare and similar platforms

An isolate platform can become a host for software-producing workloads, not only predeployed handlers. Small generated modules can sit near their tools and data, reuse platform machinery, and become callable capabilities. The compiler can itself be a WASM component of that environment. The opportunity is a common build/check/run/observe contract across local applications and managed execution—not an obligation to deploy a separate server per generated instrument.

Cloudflare's workerd README describes compatible nanoservices sharing a thread and process and connected through capability bindings. Cloudflare Code Mode documents generated programs that compose tools. Project Think describes durable execution, child agents, sessions, dynamic execution, and self-authored extensions. These are substantial precedents: the opportunity cannot be presented as "nobody has considered agents writing tools inside a runtime." [U4, U6, U9]

Hosted Workers and self-hosted workerd are not the same guarantee

At the inspected workerd revision `587bab1`, the factory source still contains null isolate and request limit enforcers under its single-tenant policy. The README also warns that workerd alone lacks the defense-in-depth required for potentially malicious code. Therefore a self-hosted design must provide appropriate outer containment, watchdogs, and resource accounting rather than inherit hosted security or quotas by implication. [U4, U5]

If a worker process must be killed to stop runaway code, every in-flight member of that process can fail. Cohort size is a performance and fault-isolation decision. Partition by trust, authority, memory budget, and acceptable shared failure—not just maximum density. Persist attempts outside the group so process loss does not erase the task history. [R1, U5]

Opportunity	Required differentiation
Point-of-need applications	A useful interactive result, not merely successful generated code execution.
Dense agent execution	Measured incremental memory and idle occupancy under realistic concurrency.
Embedded semantic checking	Pinned compiler parity and in-isolate latency, not transpilation alone.
Measured generative UI	Behavior, task completion, accessibility, and continuity—not screenshots alone.
Swarm improvement	Better accepted results at an equal complete budget, with frozen evaluation.

Potentially transformative outcomes do not require inventing every primitive. They require an integration that changes the cost, latency, or usability threshold enough to enable a new class of everyday software. The proposed workshop should be judged on that evidence, not on a platform-replacement narrative.

15 / IMPLEMENTATION CASE STUDY / OBSERVED SUBSET

Field OS: one application of the substrate

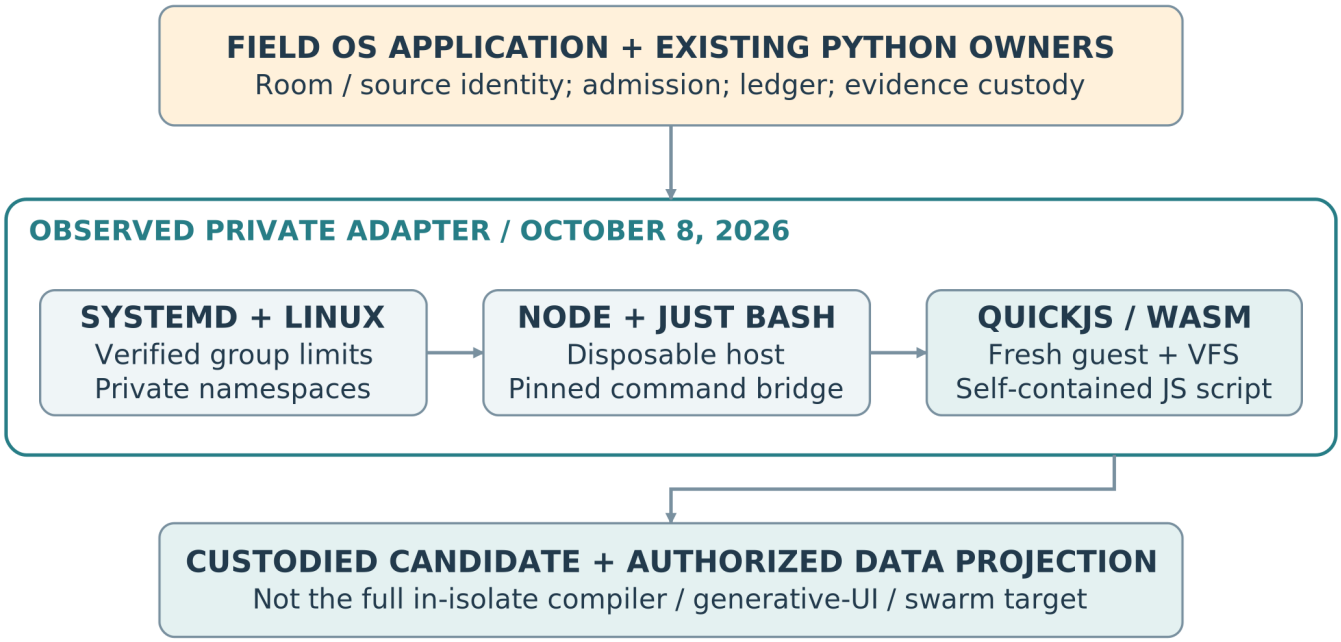


Figure 10. The checked-in Field OS adapter uses Node, Just Bash, and fresh QuickJS/WASM guests under Linux containment. It does not yet run the WASM TypeScript compiler or full swarm/UI loop in the guest. [R4–R8]

The reviewed implementation is pinned to `Akilleez-QA/field-ops` at `58d6826b2ac66d7d7ca7edd2f2e1bd869b5abe18`, dated October 8, 2026. The repository applies the larger idea to source-bound field instruments in a native Linux environment. Existing Python owners retain identity, permissions, ledger state, evidence custody, and candidate projection. [R1, R6]

The private gateway accepts a self-contained JavaScript script staged from admitted content-addressed objects. It verifies the runtime closure and current source authority. Its small adapter creates fresh QuickJS and VFS state, removes underlying tool/fetch/command ports, and returns bounded candidate JSON. The installed application is not switched to this private path. [R6, R7]

Why this case study matters beyond the domain

It supplies concrete observations about warm machinery, fresh mutable state, bounded execution, provenance, and crash recovery. Those lessons transfer to a general software substrate. It does not establish a shared V8 workerd deployment, in-isolate tsc-rs speed, generative UI usefulness, swarm convergence, or appliance-wide energy savings. [R3–R5]

The output remains a candidate. A guest cannot establish approval by returning `accepted: true`; the host's envelope retains generated, unreviewed, and unapproved status. Source access and integrity are rechecked on later projection. That distinction is equally relevant to generated business tools, coding assistants, and analytical UIs. [R5]

Recorded performance: cold and warm execution

The fixture is the complete six-case `fuseEngineeringClaims` packet: equivalent and unequal power quantities, repeated source origin, gauge/absolute pressure without assumed atmosphere, an unavailable page, and reversed inputs. The compiled program is 51,998 bytes. Native and guest outputs matched the frozen oracle and exact output hash. These are synthetic contract cases, not independently verified plant calculations. [R3]

Cold lane; 50% CPU quota	Whole wall	Payload wall	Unit CPU	Peak memory
Native Node	178 ms	28 ms	0.050 s	22.61 MiB
Just Bash js-exec	814 ms	669 ms	0.384 s	117.40 MiB

Warm lane	Quota	Median	Sample p95	Unit CPU*	Peak memory
Preloaded native function	50%	0.146 ms	0.224 ms	0.077 s	29.54 MiB
Just Bash / shared VFS	50%	24.04 ms	86.22 ms	1.021 s	163.41 MiB
Preloaded native function	100%	0.131 ms	0.182 ms	0.073 s	29.96 MiB
Just Bash / shared VFS	100%	18.64 ms	46.82 ms	1.003 s	150.62 MiB
Just Bash / fresh VFS	100%	18.65 ms	40.49 ms	0.995 s	151.93 MiB

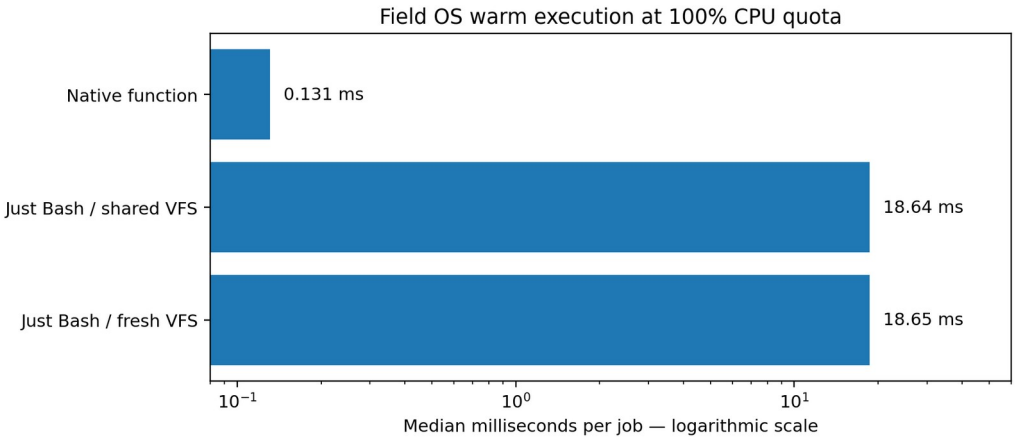


Figure 11. Warm per-job medians at 100% CPU quota, on a logarithmic axis. The native baseline is much cheaper for this reviewed function; guest value is its execution boundary and programmable environment. [R3]

*Unit CPU covers all 25 jobs: five warm-ups and twenty measured jobs. Per-job latency excludes host oracle verification; unit CPU and peak memory include startup, warm-up, and verification. Fresh VFS means recreated per job. CPU throttling occurred even in the 100% series. [R3]

The descriptive ratio 669 / 24.04 is about **27.8×**: cold payload latency versus warm shared-VFS median at the same 50% quota. It indicates amortizable adapter overhead, not a matched full-gateway, security-equivalent, or whole-system speedup. Fresh-VFS versus shared-VFS medians at 100% differ by 0.01 ms in these samples; that is not evidence of a statistically meaningful difference. [R3; derived arithmetic]

17 / FIELD OS RECEIPTS / OBSERVED

Containment, limits, and failed observations

Full-gateway case	Observed outcome	Whole wall	Group peak
Room A comparison	delta 6; schema passed	937 ms	125.2 MiB
Room B comparison	delta −3; schema passed	923 ms	124.0 MiB
Domain bundle	Six outputs match frozen oracle	984 ms	131.1 MiB
Guest child denial	Command refused; namespace checks pass	925 ms	125.9 MiB
Infinite loop	Timeout; exhausted	2,389 ms	134.9 MiB
Allocation bomb	Guest heap exhausted	1,067 ms	193.2 MiB
Output flood	External timeout; exhausted	5,230 ms	140.8 MiB

All seven final W1/W2 gateway cases passed their scoped criterion, with admission → claim → authorization → terminal events and verified empty/removed cgroups. Reopening the ledger verified 39 events; a historical retry launched no new process. These are single fresh-process observations, not a latency distribution. The flood timed out before reaching the transport cap, so it did not independently verify that threshold. [R4]

Concrete adapter envelope

The small-job profile specifies 256 MiB group memory, zero swap, 50% CPU quota, 32 tasks, and five seconds of payload wall time. VFS is bounded to 32 MiB, the stripped source/input frame to 1 MiB, native diagnostics to 64 KiB per stream, and candidate transport to 4 MiB. The first return gateway accepts at most 64 KiB of schema-checked candidate JSON. Binary input custody is checked, but this JS-only adapter does not decode it. [R6]

The W1/W2 runtime manifest covered 557 public files totaling 166,262,332 bytes, including Node and shared libraries. That installed closure is a different measurement from the compiler WASM download size or a guest's memory peak. The runtime guide records Python 3.14.7, systemd 261.2, Bubblewrap 0.12.0, and Node 26.8.1 on the observed host. [R4, R6]

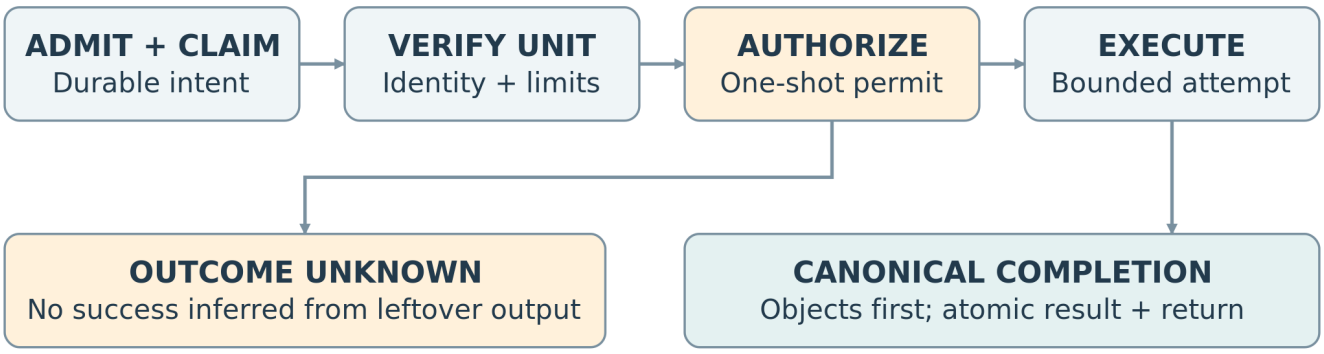
The model experiment did not succeed

The recorded 27B Ridge GGUF trial loaded in 4.14 s, but its CPU-only, two-thread request hit a 90 s client deadline without completed JSON. The unit peaked at 9.83 GiB, consumed 184.13 CPU-seconds, and closed after 94.74 s of whole supervision. This is evidence of bounded attempt/cleanup—not model quality, useful inference, or small-device fit. [R3]

For the general substrate, local inference is a separate placement and admission problem. WASM tooling and isolate efficiency do not make model weights disappear. Remote or shared inference may complement local execution; its latency, cost, permissions, and connectivity must be included in the task budget.

18 / FIELD OS RECEIPTS / OBSERVED

Recovery and test qualification



Cancel / recover only the exact owned execution group.

Waiting work persists a continuation and releases its compute slot.

Figure 12. Durable intent precedes execution. An authorized attempt may have executed even if completion was never committed. Recovery preserves ambiguity instead of inventing success. [R1, R5]

The W3/W4 return trial passed 17 named checks in 3.425 s. Room A completed in 947 ms at 127,606,784 bytes peak; Room B in 967 ms at 129,310,720 bytes. Results retained source lineage and host qualifications; cross-room access was refused, authorized stale context produced an unapplied rebind proposal, and revoked or corrupt outputs withheld payloads and pointers. [R5]

Recovery boundary	Recorded result
Killed after claim, before unit	Unknown; no redispatch
Killed before durable owned-unit identity	Passive bounded cleanup observation; unknown
Killed after permit or payload release	Exact owned closure; unknown
Killed after successful spool, before canonical finish	Unknown; no output references; spool not promoted
Active cancellation after payload release	Cancelled only after verified group closure

Five Linux recovery methods passed across six controller trials in 9.081 s. The main-based published-checkout suite recorded 180 tests in 6.898 s: 170 executed passes and 10 explicit Linux skips; opt-in recovery and containment checks were run separately. The prior broader dirty-worktree run remains failed: 2,106 tests, 17 failures, 325 errors, and 259 skips. These are different scopes, not contradictory claims of one universal green suite. [R4, R5]

Earlier W1/W2 focused regressions recorded 112 tests in 3.142 s: 107 executed passes and five Linux skips. Separate opt-in execution tests covered seven contained attempts. [R4]

The receipts cover scoped admission, strict wire parsing, custody, return races, fsync/disk-full faults, revocation, and named recovery boundaries. They do not establish physical power-loss survival, malicious same-UID protection, multi-tenant security, complete native rendering, useful generative UI, or the target swarm loop. [R4, R5]

19 / ENGINEERING BLUEPRINT / PROPOSED

Contracts and implementation interfaces

A production substrate needs a small stable interface between generated code, the compiler, the host, and the evaluator. Keep the API typed and versioned, but validate at runtime: types are not authorization and JavaScript inputs are not guaranteed to honor declarations.

Contract	Required fields or semantics
Capsule	Artifact digest; source/module map; locked dependencies; entrypoint; compiler/runtime pin; input/output/state/view schemas; capability requests; lineage.
Attempt	Task and parent candidate IDs; source/context revision; artifact digest; grant generation; deadline; resource profile; idempotency key.
Build receipt	Source/config/dependency hashes; checker identity; mode; diagnostics; emitted-byte hashes; parity/test status.
Evaluation receipt	Exact evaluator and fixtures; host metrics; seed; warm/cold mode; correctness gates; repeated samples; quality evidence; complete cost.
Activation	Accepted artifact and scope; compatible state migration; expected active revision; current grants; reviewer/policy; rollback pointer.

Illustrative capability-facing types

```
type Digest = string; // Validate format at runtime.
type Candidate = Readonly<{
  artifact: Digest;
  sourceRevision: string;
  inputSchema: Digest;
  outputSchema: Digest;
  viewSchema?: Digest;
  stateSchema?: Digest;
  requestedPorts: readonly string[];
}>;
type Evaluation = Readonly<{
  artifact: Digest;
  evaluator: Digest;
  hardGatesPassed: boolean;
  cpuMicroseconds: number;
  wallMicroseconds: number;
  peakBytes: number;
  evidenceRefs: readonly Digest[];
}>;
```

These types are a proposed interface, not exported Field OS code. The actual Field OS wire subset uses bounded integers and explicit decimal strings for physical values, rejects duplicate keys and invalid Unicode, and uses a pinned cross-language canonical encoding. Its stricter contract should not be silently widened when building a general adapter. [R6]

Generated source can embed confidential data. Transitive input/authoring scope therefore follows code, templates, caches, and outputs. Promoting a useful function does not declassify inline customer values. Reusable packages should separate data-free logic from scoped inputs and recheck access on reuse. [R1]

20 / RUNTIME OPERATIONS / PROPOSED

Operating the substrate across hosts

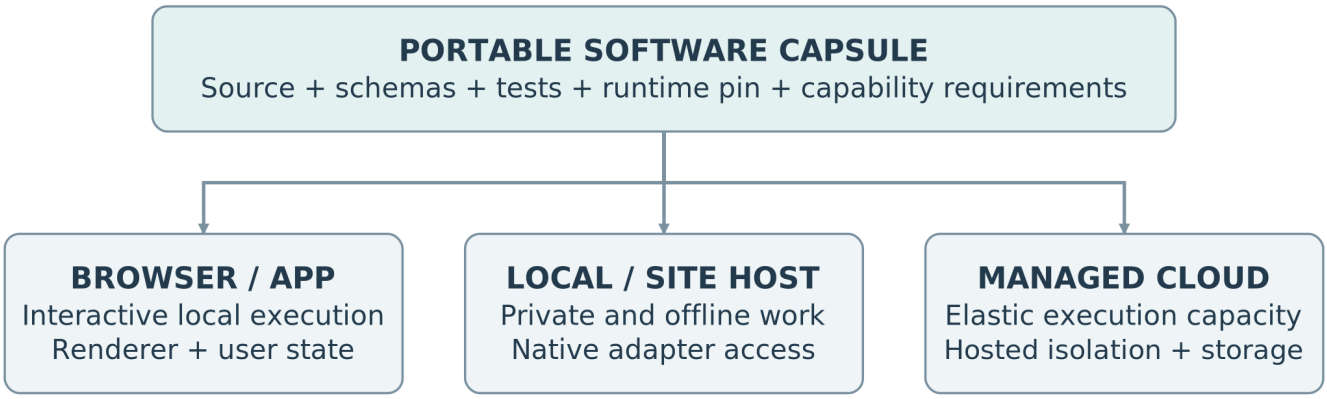


Figure 13. The portable unit is a capsule and checkpoint contract—not an assumed migratable live heap. Host adapters determine storage, UI, networking, native tools, and enforcement.

Scheduling and backpressure

Start with explicit limits on runnable queue size, active compiler instances, guest concurrency, and total memory. Separate UI interaction from background candidate search. Throttle speculative swarms before capture, user input, or durable-state work is affected. Parallelism should follow measured bottlenecks; aggressive native checker threads can reduce the number of candidates that fit. [R1, U1]

Cancellation and containment

Use an external deadline mechanism capable of terminating the actual execution unit. A timeout scheduled on the same blocked event loop cannot stop a synchronous CPU loop. Bound compiler input, filesystem size, decompression, output, host bridge traffic, and broker amplification. Keep secrets and canonical state outside candidate namespaces; read-only mounting of a sensitive host file still exposes its contents. [R1, R7, R8]

Publication and lifecycle

Store immutable outputs before committing references. Activate by a guarded revision change; do not replace active code during partial compilation. On failure, keep the incumbent and a bounded diagnostic. Reconstruct waiting work from durable references after eviction or restart. Treat caches as optional acceleration, not a correctness dependency. [R1, R5, U7]

Collect per-phase latency, CPU, peak and retained memory, model usage, bytes copied, cache hit/miss, checker mode, active queue pressure, failure cause, and artifact/evaluator identities. After teardown, verify that the intended resources were actually released. Count renderer and broker work outside the guest group when reporting whole-system efficiency.

21 / VALIDATION PLAN / NOT YET RUN

The experiment that would establish the full claim

The next decisive experiment is not another isolated benchmark. It is a matched, end-to-end campaign: from intent or source delta to an accepted working capability, including generated UI and budgeted candidate search.

Arm	Purpose
A. Existing warm native/declarative baseline	Control for the simplest reviewed solution and an optimized existing toolchain.
B. Current private Node / QuickJS adapter	Preserve the known behavior, receipts, and containment baseline.
C. Strict in-isolate WASM compiler + evaluator	Test the complete embedded build/check/run path on the chosen host.
D. Compiler Worker + Dynamic Worker execution	Compare hosted loader topology rather than pretending it is identical to C.
E. Native/container fallback where required	Represent real compatibility needs and compare equivalent workloads.

Freeze the comparison before running it

Pin compiler and runtime builds, SDK declarations, dependency graphs, task packets, evaluator code, hardware, host limits, warm/cold rules, and acceptance thresholds. Use identical fixtures and diagnostic/behavioral requirements. Include small and medium projects, deep-type stress, UI state migrations, varied data sizes, deliberate failures, and many waiting tasks. Measure candidate populations such as 1, 4, and 16 subject to safe admission—not as a promise those levels fit every host.

Measure both the candidate and the search

Record first-load latency, warm type-check/emit, complete accepted-artifact latency, interaction latency, CPU-seconds, peak and retained memory, queue time, model tokens/cost, correctness, reviewer effort, and improvement per total search budget. Swarm comparisons should use the same total resource budget and held-out tasks; faster evaluation alone does not establish better final outcomes.

Suggested prospective acceptance rule

Require identical hard correctness and authority gates, successful cancellation/recovery, and no hidden resource growth. Then choose an explicit product target before observing results—for example, at least 2× lower median time to an accepted small capability, or materially more accepted improvements at the same complete cost. These are proposed targets, not current measurements. Report uncertainty and failed tasks alongside the medians.

THE MOST INFORMATIVE DEMONSTRATION

A user requests a working interface; bounded workers generate alternatives; the WASM compiler and tests reject weak candidates; independent measurements select a better version; the UI updates without losing state; the same capability survives restart and remains useful offline where its dependencies allow.

Stop or narrow the approach if diagnostic parity fails, host memory erases the density benefit, evaluator cost dominates, native/declarative tools are superior, or users cannot reliably understand the generated result. A negative experiment can change the implementation without invalidating the broader product goal.

What changes if the complete loop works

Applications become compositions of generated capabilities

Instead of selecting a prebuilt application that approximately matches a task, a user could receive a tailored working surface inside an existing environment. Stable domain objects, capability contracts, and renderer primitives provide continuity; generated code supplies the task-specific behavior. Field OS is one setting. The same mechanism could support analytical workbenches, coding tools, support operations, internal workflows, education, simulation interfaces, or personal computing.

The environment becomes a software workshop

The compiler is no longer only a developer workstation utility. As a portable WASM component, it can participate in an application's runtime. The runtime becomes a place where software is proposed, checked, rehearsed, compared, and retained. Dependency preparation and host interfaces still matter, but they become shared infrastructure rather than repeated setup for each small tool. [U2, U3]

Improvement becomes an accountable loop

Generative UI and swarm search reinforce each other: software candidates can vary both behavior and interaction, while fixed evaluators measure whether the changes actually help. Faster checking and lower execution overhead permit more experiments within a budget. That does not guarantee convergence or correctness; it makes a disciplined search process more affordable.

The model is used where it adds value

Reasoning chooses and revises a strategy. Compiled or interpreted code performs repeatable operations. A successful program becomes an asset that can run without repeating the original reasoning. This moves some work from recurring inference into reusable software. It is an economic hypothesis to test at the whole-task level, not an assertion that model cost vanishes.

The claim that can be made today

The local receipts establish warm guest reuse, fresh-state execution, guarded admission, candidate custody, and named recovery behavior. Upstream reports establish a fast native Rust TypeScript checker and a source-buildable compact WASM interface. Cloudflare supplies relevant dynamic execution and agent primitives. Together they support a serious engineering path to the proposed substrate. They do not yet establish its end-to-end speed, resource advantage, UI usefulness, or swarm-search benefit. [R3–R5, U1–U10]

BOTTOM LINE
The opportunity is not merely "a faster coding agent." It is to make the creation and measured improvement of working software a routine capability of the application runtime itself. That could be a category-level change if the complete loop clears its technical and user-value tests.

Implementation priority: prove a small, fully in-isolate compiler/evaluator loop first; attach one useful generative interface; add independent measurement and bounded variant search; then scale only the dimensions whose complete economics have been demonstrated.

23 / REFERENCES / REPOSITORY EVIDENCE

Sources and reproducibility

Private repository references require access to Akilleez-QA/field-ops. Public project references are pinned to inspected commits where possible. Live documentation was read on October 14, 2026. All numerical claims in this document are attributed as recorded, upstream-reported, or derived; proposed targets are not observations.

[R1] Field OS Worker Fabric v1
Design dated October 7, 2026; private subsets observed October 8. Architecture, authority, lifecycle, budgets, source lineage, and proposed acceptance matrix.

[R2] Worker fabric design council
Alternatives and rationale: native/declarative baseline, generated QuickJS jobs, conditional workerd services, runtime lifetime separation.

[R3] Main-rig execution receipt
October 8, 2026. Cold/warm timings, unit CPU and memory, six-case oracle, and failed 27B local-model attempt. Raw machine receipts were not rerun here.

[R4] Private W1/W2 receipt
Seven full-gateway cases, contained adversarial execution, runtime closure size, strict wire tests, durability faults, and broad-suite failures.

[R5] Private W3/W4 receipt
Candidate return, 17-check gateway trial, six recovery boundaries, main-based test results, and retained qualifications.

[R6] Worker fabric runtime guide and gateway
Current private adapter limits and status. Gateway implementation: src/fieldops/worker_fabric_gateway.py at the same repository revision.

[R7] Trusted JavaScript guest bridge
Fresh QuickJS/VFS setup; removed execution, network and tool ports; bounded output transport.

[R8] Native execution supervisor
Runtime inspection, cgroup/systemd readback, namespace and owned-process supervision.

[R9] Operational instrument binding v1
Exact source/owner identity, room persistence, typed presentation boundaries, and limits on native UI/authentication qualification.

Key frozen identities

Field Ops implementation: `58d6826b2ac66d7d7ca7edd2f2e1bd869b5abe18`. Main-rig shared domain bundle: 51,998 bytes, SHA-256 `a82e307347259e8be61e6762d3408a12ee8245cea1be907d2dfe484e4d62231e`. Exact six-case output SHA-256: `1c5facb29b24ff5f5d7f8c22e8fa44763b8ce6c70ed5ca97df4bf56d86b1033d`. [R3, R4]

The private experiment directories and raw receipts remain on the original machine. This document is a source-backed synthesis, not a new test run, deployment receipt, or independent physical validation. No repository or application configuration was changed by this document task.

24 / REFERENCES / PUBLIC PRIMARY SOURCES

Upstream sources and evidence boundaries

[U1] ts-rust / tsc-rs README and benchmark methods
Read October 14, 2026. Native tsc-rs 0.6.0 benchmark measured October 13 on 16-vCPU Blacksmith runners. Includes compiler versions, diagnostic parity, fast-mode tradeoffs, and raw-results links.

[U2] ts-rust WASM interface
Source-build instructions; 4.2 MB module; in-memory filesystem; compile-once/instantiate-per-run; single-threaded and stack limitations; publication disabled.

[U3] ts-rust runtime-independent WASM core
Host filesystem bridge, WASI imports, module/instance integration, and dependency surface.

[U4] workerd runtime README
Nanoservices, capability bindings, server-first scope, and explicit warning that standalone workerd is not a hardened sandbox.

[U5] workerd worker-factory implementation
Inspected current revision on October 14, 2026. Single-tenant null isolate and request limit enforcers; code moved from the older server implementation.

[U6] Cloudflare Code Mode documentation
Current official documentation read October 14, 2026. Generated code composes typed tool operations; experimental status.

[U7] Cloudflare Dynamic Workers API reference
Updated October 12, 2026; read October 14. load/get behavior, cache identity, module formats, bindings, and outbound defaults.

[U8] Cloudflare Workers WebAssembly API
Official API documentation read October 14, 2026. WASM execution and precompiled-module instantiation requirements.

[U9] Cloudflare Project Think
Official engineering/product description read October 14, 2026. Durable execution, sub-agents, sessions, dynamic execution, and self-authored extensions; relevant prior art, not a benchmark of this proposal.

[U10] Cloudflare Dynamic Workers Playground
Official example read October 14, 2026. Runtime TypeScript bundling, Dynamic Worker execution, caching, and observability; not evidence of full tsc-rs checking.

Availability, limits, and runtime behavior can change. Revalidate the exact build and target environment before implementation. Upstream-reported performance and security statements are not silently promoted into guarantees of this substrate.